

02/18/2026

schwarz digits

TypeScript 101

A practical workshop to learn the basics of TypeScript
by Mark Brockhoff





Who am I?

Name:

Mark Brockhoff

Position:

Full Stack JS/TS Engineer at [Schwarz Digits](#)

Experience:

> 7 years as a developer, > 6 of them using TypeScript

Hobbies:

- 🎷 Playing saxophone in a band
- ✈️ Traveling
- 💡 Small embedded projects e.g. IoT, Home automation

Agenda

- 01 What is TypeScript and why does it exist?
- 02 Basics of the TS programming language
- 03 Hands on session 1
- 04 Working with async / await
- 05 Hands on session 2
- 06 Type narrowing & error handling
- 07 Hands on session 3
- 08 Wrap up / Q&A

What is TypeScript and why does it exist?



A look at the history of TypeScript

In the early 2010s web apps got more and more complex with Apps like **Gmail** and **Google Maps**

- JavaScript was designed for small scripts not huge enterprise applications
- The missing type system lead to bad scalability as the code base grew larger, e.g. due to the lack of good tooling
- Errors might only be caught at runtime possibly for the end user

Anders Hejlsberg at Microsoft designed TypeScript to solve this problem

- TypeScript is designed as a “Superset” of JavaScript -> JS code is also valid TS code
- Open Source from the beginning, since the release in 2012
- Google later decided to rewrite v2 of the popular framework Angular in TypeScript

Typescript today

- According to the 2025 Stack Overflow survey TS is 6th most popular programming language, slowly catching up to number 1 -> JS
- Thanks to a huge community, ecosystem and tooling TS is the de-facto standard for web but also heavily used for servers as well as desktop and mobile applications

What's the advantage of TS?

JavaScript

```
const foo = 'bar';  
foo = 42;
```

TypeScript

```
const foo: string = 'bar';  
foo = 42;
```

- => TypeScript allows us to catch errors at build time
- => It allows us to model our code in a way where undefined states can't exist
- => It improves our developer experience with great tooling e.g. intellisense

What is TypeScript and why does it exist?

How powerful is TypeScript's type system?

TypeScript's type system is basically a functional programming language by itself. Allowing you to build really smart types, e.g.:

- Allowing only capitalized strings to be passed to a function ([Example](#))
- Having a variable which could have multiple different types ([Example](#))
- Making sure a regex pattern is valid (e.g. [ArkRegex](#))
- Adding autocomplete to GraphQL or SQL queries and typing the response (e.g. [gql.tada](#) or [Drizzle](#))
- Executing a bubble sort in the type system (e.g. [Github](#))
- Or even running Doom only using types: [Youtube: Typescript types can run doom](#)



Basics of the TypeScript programming language



Syntax basics

Runtime

- [Constants and variables](#)
- [Primitive values](#)
- [Objects and arrays](#)
- [Functions](#)
- [Conditions & Loops](#)

Build time

- [Defining types](#)
- [Deriving types](#)
- [Composing types](#)

The official TypeScript docs contain very useful cheat sheets in case you want to look up some syntax:
<https://www.typescriptlang.org/cheatsheets/>

Let's get practical

Hands on session 1

03

Example App & repo setup

1. Clone the Repository from Github
2. Open it in an IDE / text editor of your choice
3. Open a terminal and install required dependencies (Node JS should be preinstalled):

```
npm install
```

4. Start the app by running:

```
npm run dev
```

5. Your browser should open `http://localhost:5173/`
(You should see a “not implemented” error)



Who won the UEFA Champions League in 2016?

FC Bayern Munich

Atletico Madrid

Real Madrid C.F.

Manchester City F.C.

Next Question

Task 1

Implement the static question provider

=> We'll have a Q&A after each Task

Task 1

Shuffle the answers
of the question by
implementing the
Fisher-Yates shuffle
algorithm



Working with async / await



Promises

Demo

- Promises are values which will be computed concurrently -> not in parallel
- Functions marked with the **async** keyword always return promises
- The **await** keyword can be used to stop code execution until the given promise is fulfilled
- Promises can **reject** e.g. if something inside them throws an exception
 - Be aware that every use of await could throw

Fetching questions from an API

Hands on session 2

05

Task 2

Fetch questions dynamically from an API



Type narrowing & error handling



Type narrowing

Demo

- When working with polymorphic data or data from untrusted sources we need to **validate** it at runtime to ensure it matches the type we expect
- In TS this is done by **narrowing** the type of a value through conditions
- If the value is too dynamic, e.g. if it's an array, a **type guard** needs to be created to perform this validation for TS
- In larger projects it may be worth it to add a validation library to make this simpler.
E.g.: [Zod](#), [ArkType](#) or [Valibot](#)

Error handling

Demo

- TS supports **exceptions**, if the program runs into an issue it can **throw** an instance of the **Error** class
- Thrown errors can be caught by using a **try/catch** block
- It's possible to create **custom errors** by extending the Error class, e.g. to store custom data about the error inside them
- When handling an error inside a catch block, **narrowing** is required to access its properties

Type safety everywhere

Hands on session 3

07

Task 3

Use type narrowing to validate the API response



Task 3

Add error handling and retries



Wrap up / Q&A

08

Good reads for recap / to go further

- [Bad practices in TypeScript](#)
- [The TypeScript Handbook](#) (part of the official TS docs)
- [Total TypeScript Essentials by Matt Pocock](#)
- <https://learntypescript.dev/>

Feedback

Please take a short moment to fill out the survey. It's 100% anonymous and helps me improve, thank you.



Let's stay in touch



schwarz digits

<https://schwarz-digits.de>